

Guía metodológica y de implementación

Flujo de trabajo con Vitis HLS, Vivado ML y Vitis IDE

Daniel Orlando López Vargas

21 de abril de 2026

Índice general

0.1.	Verificación de proyecciones Q,K y V en primera capa únicamente	2
0.1.1.	Generación de vectores de entrada y salida de referencia (golden) del modelo BETO	2
0.1.2.	Extracción de datos de referencia en precisión flotante	2
0.1.3.	Cuantización de pesos por canal a precisión entera de 8 bits	2
0.1.4.	Validación software de la cuantización	2
0.1.5.	Preparación de vectores de entrada/salida para la FPGA	2
0.1.6.	Coverción datos hls para pre-adequación a la simulación FPGA . . .	3
0.2.	Prototipo HLS de GEMM	3
0.3.	Prototipo integración del IP HLS	4
0.4.	Desarrollo de la aplicación bare-metal	5
0.5.	Carga y debug de la aplicación bare-metal por medio de xsdb . .	6

0.1. Verificación de proyecciones Q,K y V en primera capa únicamente

A continuación se presenta la función de cada apartado como resumen breve de función de los scripts en el repositorio

0.1.1. Generación de vectores de entrada y salida de referencia (golden) del modelo BETO

Se usan 3 frases de entrada como ejemplo para generar los vectores de entrada en int64
Script asociado: `beto_make_golden.py`

0.1.2. Extracción de datos de referencia en precisión flotante

Se extraen en formato punto flotante (float32) de los vectores de entrada como proyecciones lineales Q,K y V (para cada texto) de la primera capa o capa 0 de Beto casad los pesos y bias $W_q, W_k, W_v, b_q, b_k, b_v$.

Script asociado: `dump_qkv_layer0.py`

0.1.3. Cuantización de pesos por canal a precisión entera de 8 bits

Convierte a un formato rígido (por eso se usa el término "congelar") de float32 a int8 (que para FPGA es el formato reducido necesario) los pesos W de la extracción de datos sección A.

Script asociado: `freeze_qkv_weights_int8.py`

0.1.4. Validación software de la cuantización

Se comparan los resultados dados por la extracción de datos en precisión flotante con una simulación con entradas cuantizadas int 8 para luego descuantizar al final y comparar ambos en float32 y obtener diferentes parámetros principalmente métricas de error.

Script asociado: `quant_test_QKV_layer0.py`

0.1.5. Preparación de vectores de entrada/salida para la FPGA

Cuantiza por tensor en int 8 las entradas X de la extracción inicial de datos sección A y usa los pesos cuantizados por canal de la sección C para multiplicarlos con X de entrada cuantizados por tensor int 8 en éste mismo paso de los datos sacados de la sección A de la extracción de datos.

Lo primero para que sirvan de entrada en la implementación real en la FPGA y lo segundo para tener una comparación posterior a la implementación

Script asociado: `prepare_fpga_io_layer0.py`

0.1.6. Conversión datos hls para pre-adequación a la simulación FPGA

Toma los pesos de la sección C y los convierte de `.npy` a `.bin` , así mismo toma los resultados de la sección E y realiza el mismo procedimiento de conversión.

Script asociado: `export_bins_for_hls.py`

0.2. Prototipo HLS de GEMM

Se hace uso de la herramienta Vitis hls, haciendo uso previamente de dos códigos `.cpp` para la respectiva simulación, los cuales son `tb.cpp` y `gemm_int8.cpp` y se inicia vitis hls y se siguen los siguientes pasos:

- **Paso 1(Figura2):** Se le asigna el nombre al proyecto en éste caso `gemm_qkv`
- **Paso 2(Figura3):** Se le asigna el nombre a la función superior en éste caso `gemm_int8` y se añade el archivo de diseño `gemm_int8.cpp` buscando en la ubicación correspondiente.
- **Paso 3(Figura4):** Se añade el archivo de test en éste caso llamado `tb.cpp`
- **Paso 4(Figura5):** Posterior a abrir clicar los puntos de `part selection` se abre esta pestaña y se selecciona la referencia del dispositivo en éste caso el fpga de código de referencia `xck26-sfvc784-2LV-c`
- **Paso 5(Figura6):** SE configura la solución, nombre, periodo del reloj y demás y se finaliza
- **Paso 6(Figura7):** Se espera la iniciación hasta que semuestre a la izquierda los paneles donde se puede ver como quedó configurado el proyecto.
- **Paso 7(Figura8):** Se inicia la simulación C y espera a que se realice
- **Paso 8(Figura9):** Ahora se realiza la síntesis C, se espera a que termine y se guardan los resultados pertinentes.
- **Paso 9(Figura10):** Para verificar homogeneidad con la síntesis en C y el RTL exportable se realiza la síntesis Co-simulación C/RTL se espera un mayor tiempo que en anteriores pasos hasta que termine.

- **Paso 10(Figura11):** Se exporta el RTL en la carpeta correspondiente.
- **Paso 11(opcional)(Figura12):** Es opcional debido a que al ser el prototipo no se requiere

De esta manera queda finalizado el prototipo RTL por medio del programa vitis hls

0.3. Prototipo integración del IP HLS

Después de exportar el archivo `gemm_int8_vivado_ip.zip` con Vitis hls se abre el programa Vivado IDE y siguiendo los siguientes pasos:

- **Paso 1(Figura13):** Después de darle nombre al proyecto se configura inicialmente así.
- **Paso 2(Figura14):** Se escoge el nombre de la placa fpga por referencia o nombre comercial.
- **Paso 3(Figura15):** En el catálogo ip se configura la ruta del repositorio IP.Y aquí se muestra la ruta donde se encuentra el archivo RTL descomprimido.
- **Paso 4(Figura16):** Se verifica que en esta sección que se reconozca el gemmint8
- **Paso 5(Figura17):** Se crea en IP integrador un nuevo bloque de diseño en este caso se llama `prototipo_1`
- **Paso 6(Figuras18,19,20,21):** Se van agregando todos componentes necesarios
- **Paso 7(Figuras22):** Se usa `Run Block Automation` cuando se vea necesario y si se conoce su funcionamiento.
- **Paso 8(Figura23):** Y se realiza doble-click derecho en los bloques que se quieran o requieran configurar.
- **Paso 9(Figura24):** Se realizan las configuraciones y conexiones correspondientes hasta llegar a el siguiente diseño circuital.
- **Paso 10(Figura25):** Se realiza `File → Save Block Design` y se verifica en esta sección si se creó el archivo `prototipo_1.bd`
- **Paso 11(Figura26):** Click derecho en el archivo el paso anterior y `Generate output products` y se espera a que termine un tiempo considerable dependiendo máquina.
- **Paso 12(Figura27):** Click derecho en el archivo el paso 10 y `Create HDL Wrapper . . .` lo que es instantáneo.

- **Paso 13(Figuras28,29,30):** Se continua con la síntesis y la implementación con la misma configuración hasta que se genere el esquema del `device`.
- **Paso 14:** Se corre ahora el `Generate bitstream`
- **Paso 15(Figura31):** Se exporta la plataforma Hardware en formato `xsa` en la carpeta correspondiente y es la conversión del `.bit` del paso anterior.

0.4. Desarrollo de la aplicación bare-metal

En esta sección se explica el paso a paso del desarrollo SW del proyecto para comprobar el prototipo base:

- **Paso 1(Figura32):** Se crea un nuevo proyecto con una nueva plataforma de hardware XSA y se escoge de la dirección del último paso de la sección anterior.
- **Paso 2(Figura33):** Se especifican los detalles del proyecto, agrega nombre del sistema y nombre del proyecto, se escogen aplicaciones asociadas.
- **Paso 3(Figura34):** Se configura el dominio del proyecto de la respectiva forma.
- **Paso 4(Figura35):** Se escoge la plantilla (template) específica del proyecto que en este caso es `Empty application (C)`
- **Paso 5:** Mediante el uso de un script en python que realiza la conversión de archivos `.bin` a arreglos C (arrays) para ser exacto 7 donde 4 son entradas y pesos de cada proyecciones (X,Wq,Wk,Wv) en `int8_t` y 3 que son las referencias (Qref,Kref,Vref) en `int32_t`
Script asociado: `gen_embed_golden_case.py`
- **Paso 6(Figura36):** Los archivos generados en el anterior paso `.c` y `.h` se agregan de la carpeta correspondiente a la carpeta `src` del proyecto y se verifica que la estructura del proyecto en vitis se vea de ésta forma.
- **Paso 7:** Se procede a crear el código de `main.c` para la puesta en prueba de funcionamiento sobre la fpga por medio de la herramienta `xsdb`.
- **Paso 8:** Se construye el programa y se espera que termine la creación del programa `gemm_beto_app.elf`

0.5. Carga y debug de la aplicación bare-metal por medio de xsdb

Para esta etapa de evaluación del software en el hardware (fpga kv260) se procede a conectar via usb la placa y a energizarla para por medio del xsdb (Xilinx software debugger) que maneja el lenguaje tcl.

Los comandos se guardan en la carpeta scripts y al correrlos en xsdb nos dan el éxito o fallo global de la ejecución, errores por corrida, detalle del primer error detectado, latencia por corrida, latencia por caso, latencia total y salidas-resumen de verificación.

Documento de texto asociado: `xsdb_tlc_commadns.txt`

Anexos

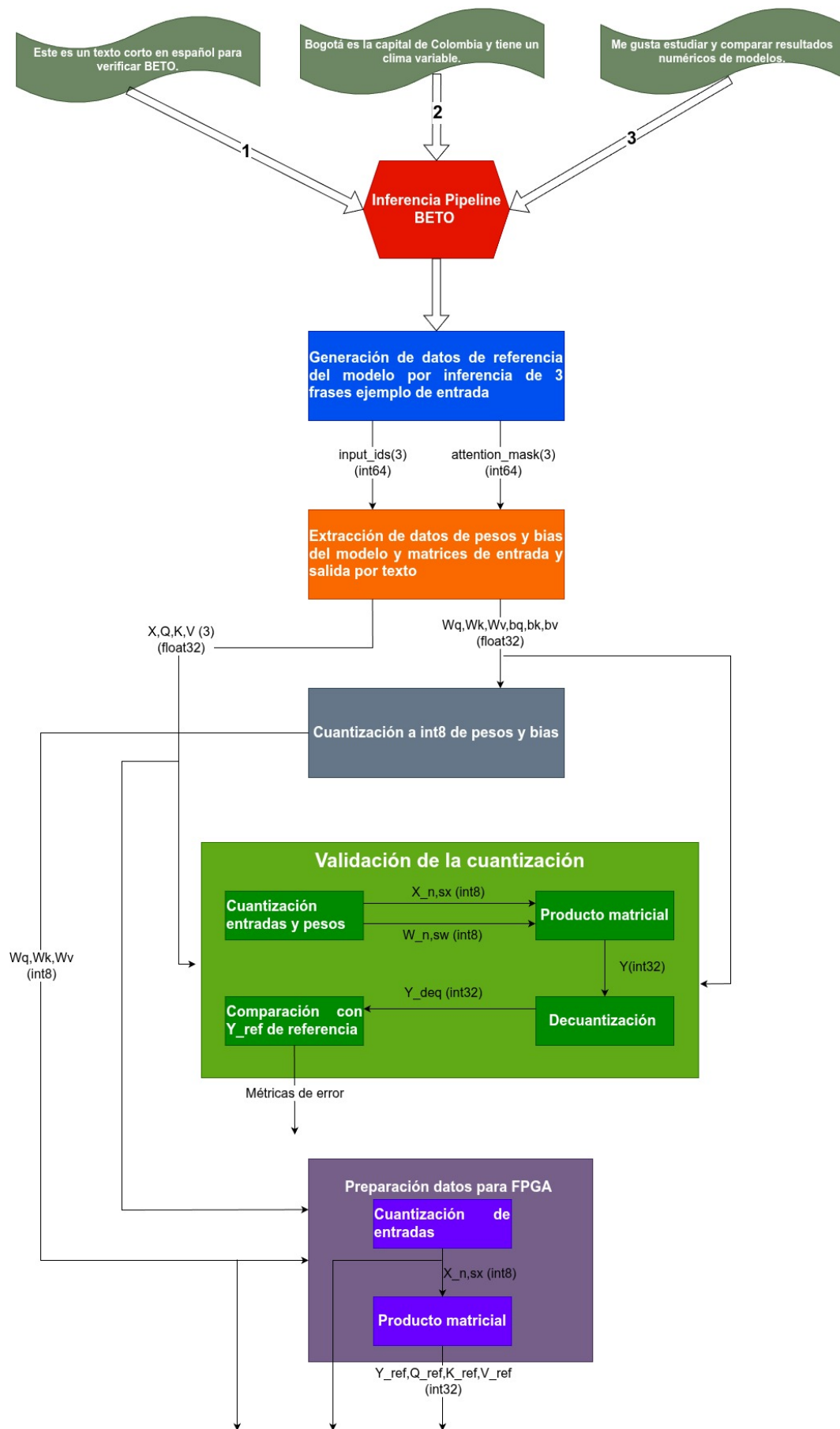


Figura 1: Flujo de datos comparativos

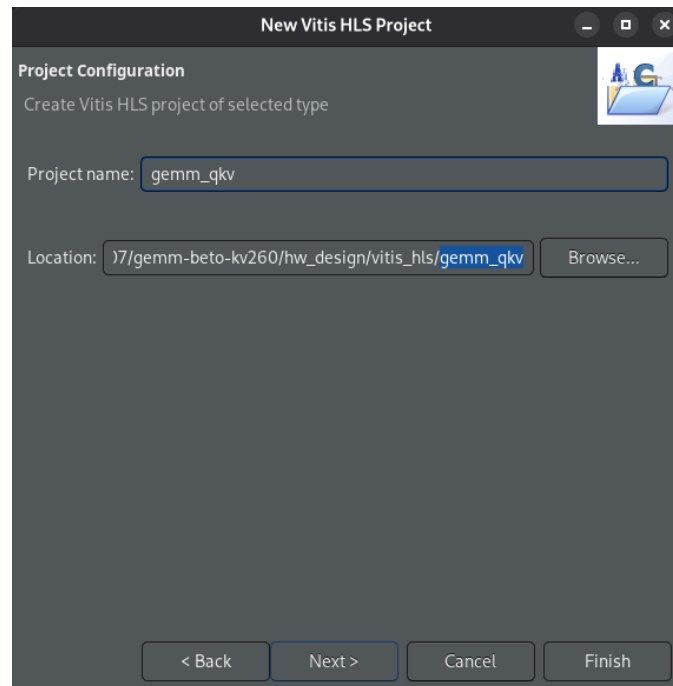


Figura 2: Paso 1 Vitis hls

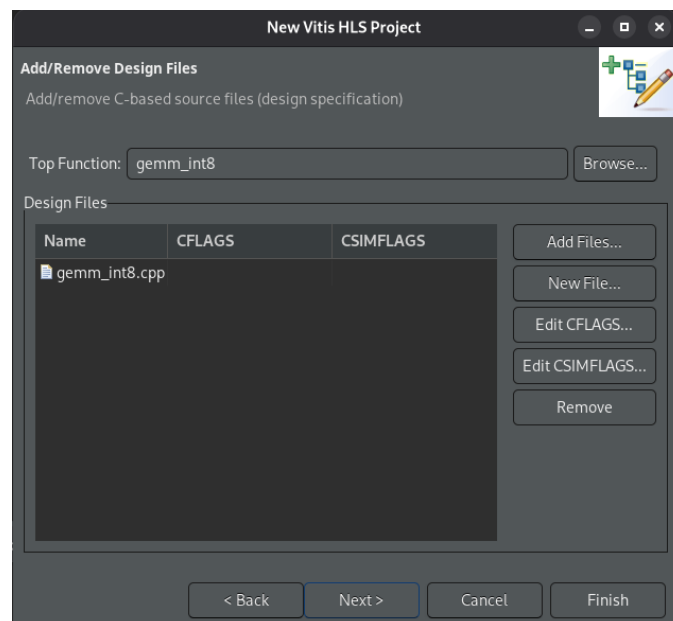


Figura 3: Paso 2 Vitis hls

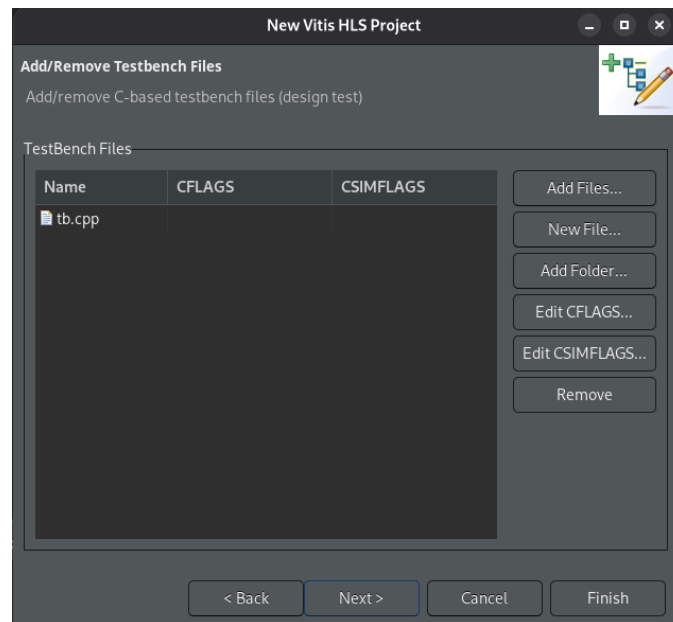


Figura 4: Paso 3 Vitis hls

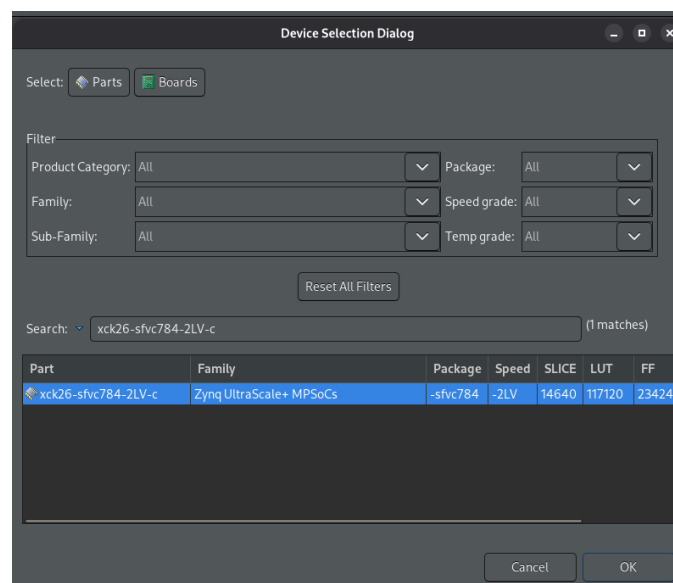


Figura 5: Paso 4 Vitis hls

New Vitis HLS Project

Solution Configuration
Create Vitis HLS solution for selected technology

Solution Name:

Clock
Period: Uncertainty:

Part Selection
Part:

Flow Target
 several options for the selected flow target" data-bbox="445 495 695 505"/>

Figura 6: Paso 5 Vitis hls

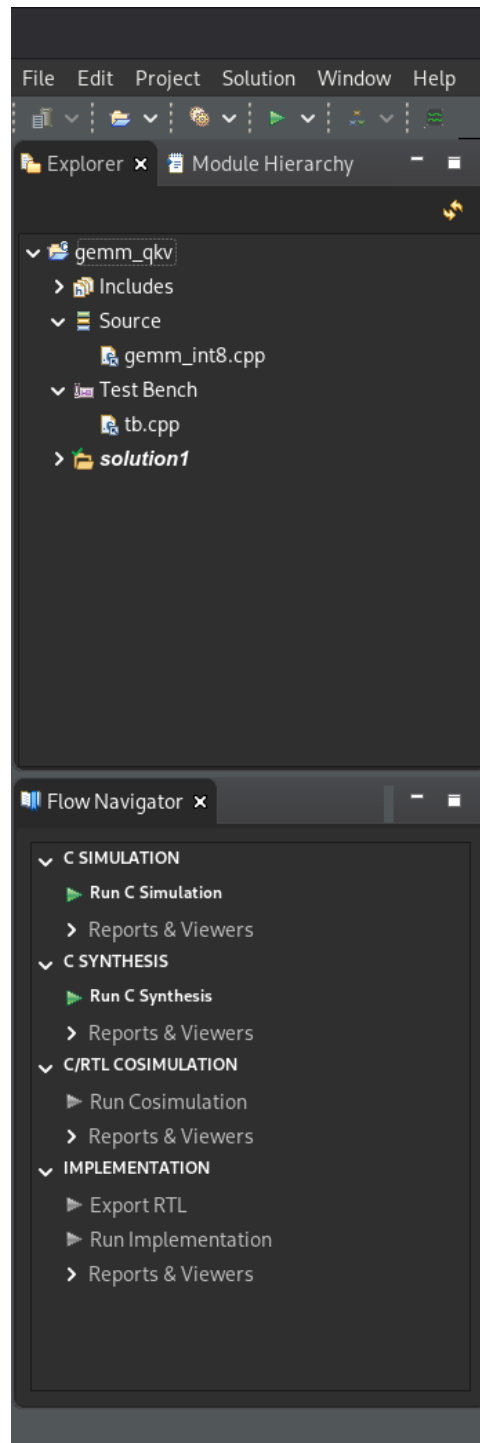


Figura 7: Paso 6 Vitis hls

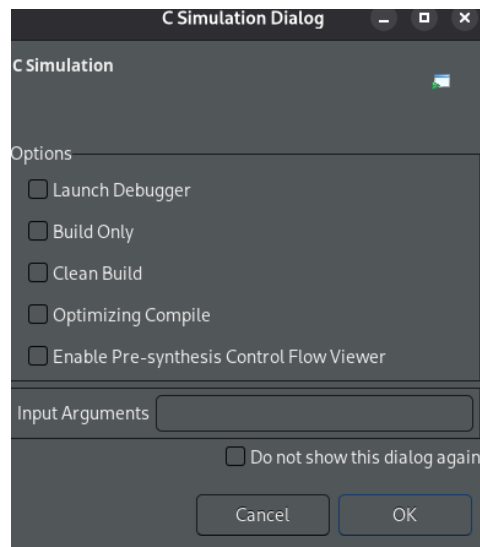


Figura 8: Paso 7 Vitis hls

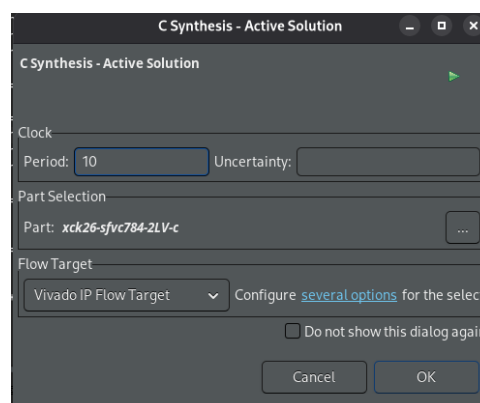


Figura 9: Paso 8 Vitis hls

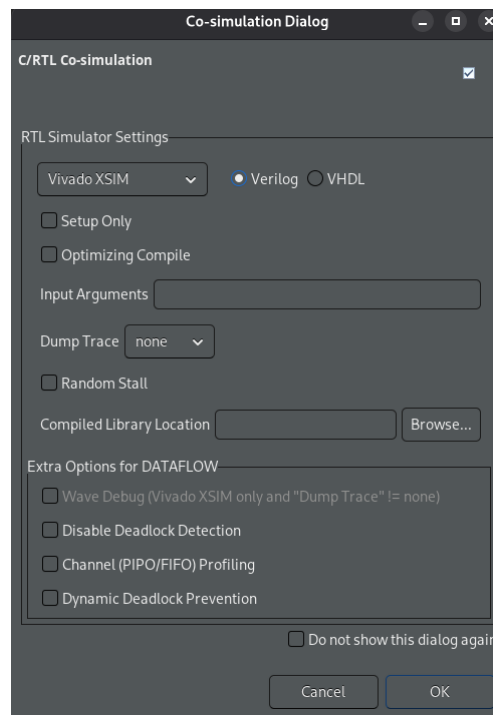


Figura 10: Paso 9 Vitis hls

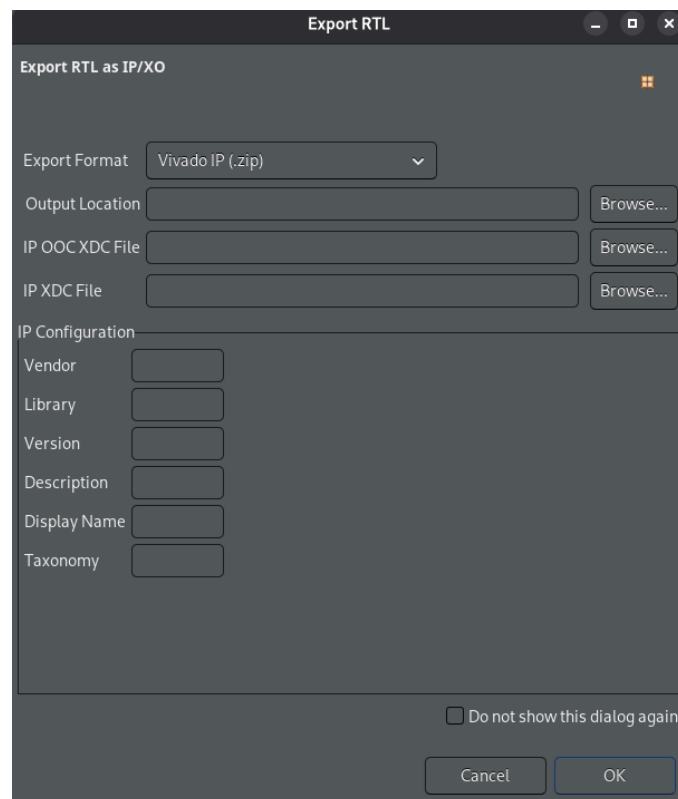


Figura 11: Paso 10 Vitis hls

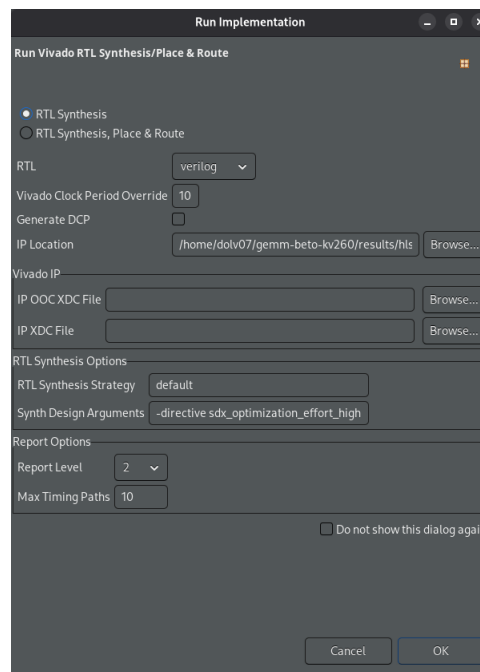


Figura 12: Paso 11 Vitis hls

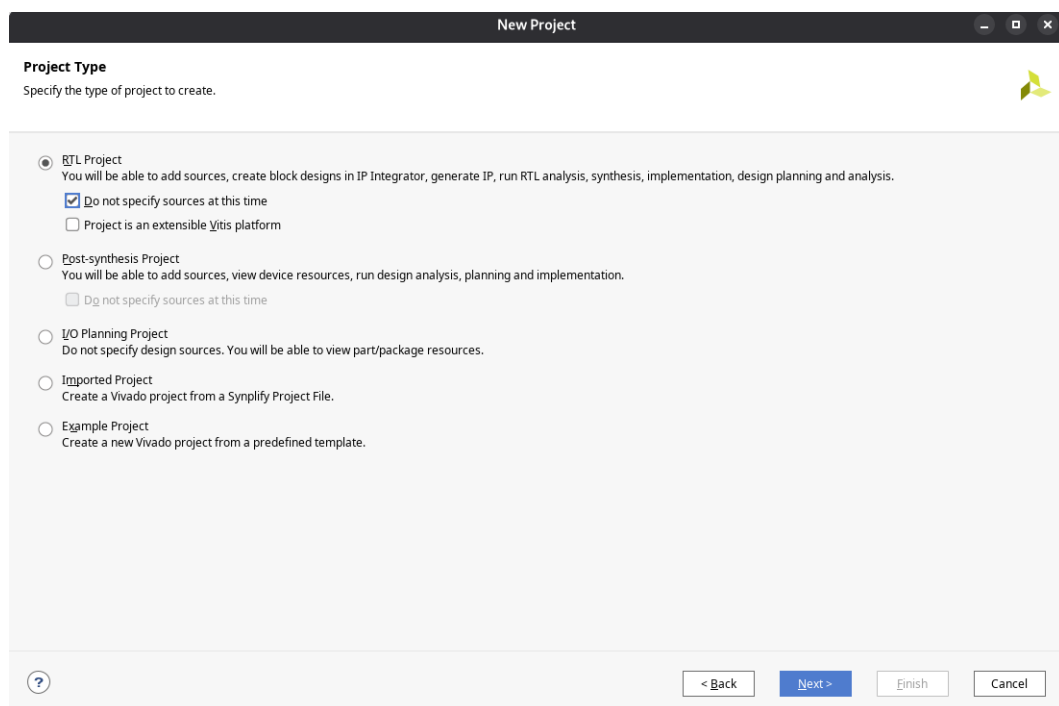


Figura 13: Paso 1 Vivado ML

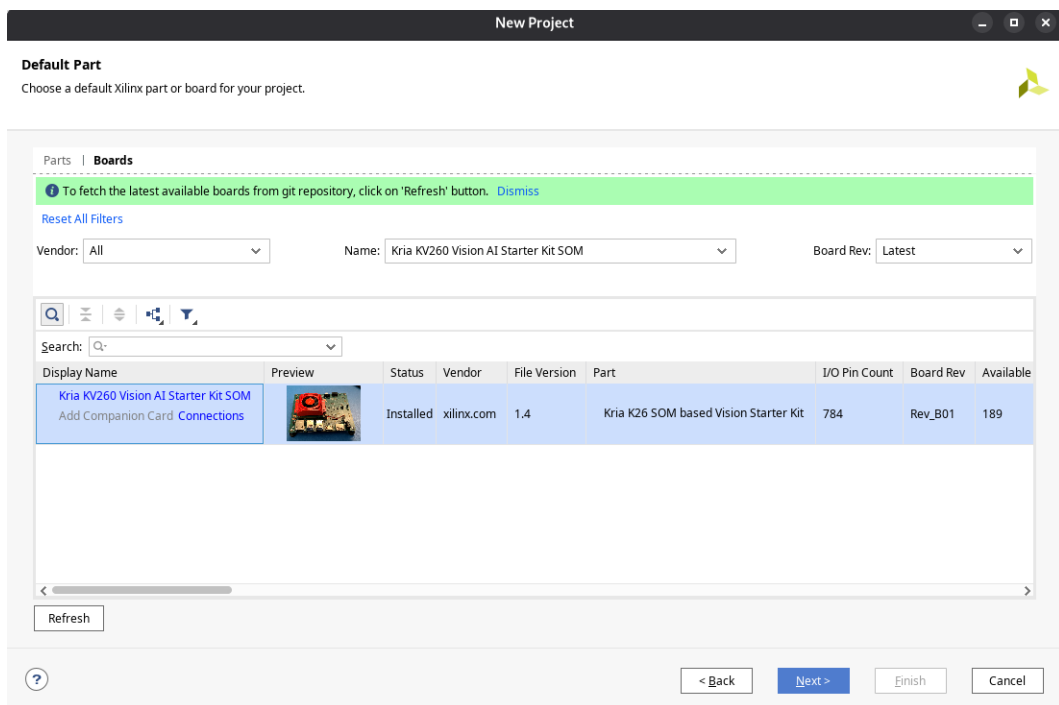


Figura 14: Paso 2 Vivado ML

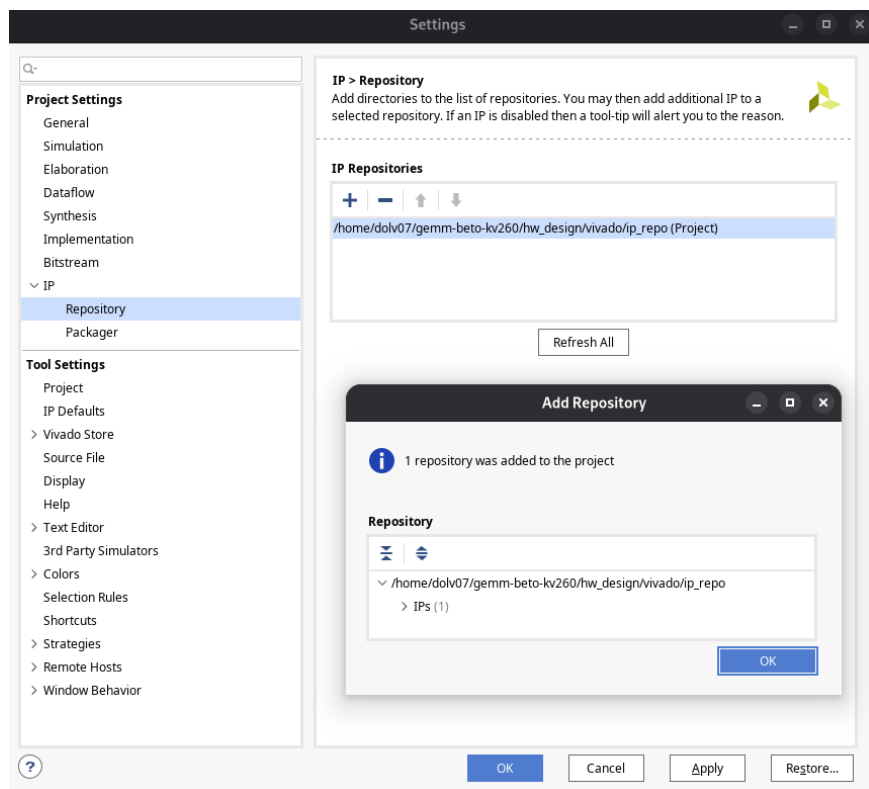


Figura 15: Paso 3 Vivado ML

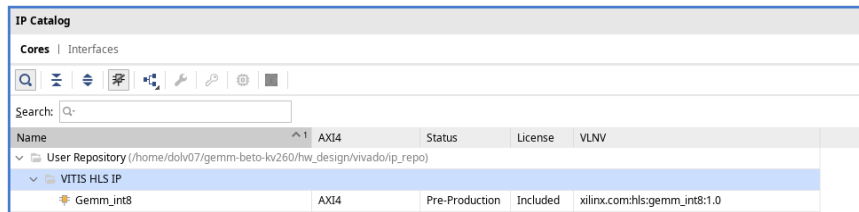


Figura 16: Paso 4 Vivado ML

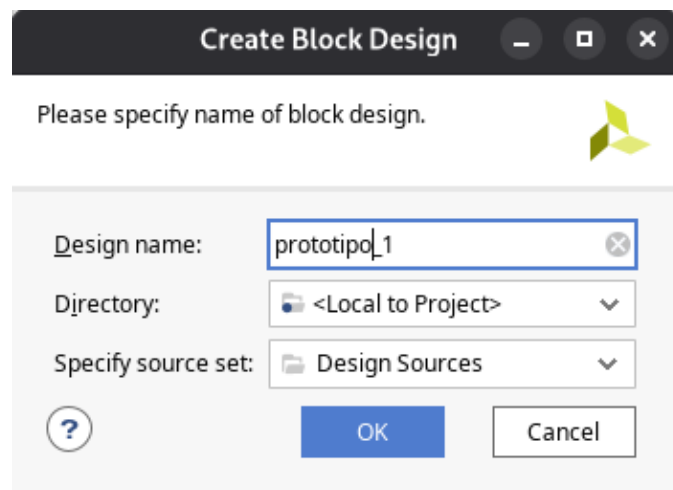


Figura 17: Paso 5 Vivado ML

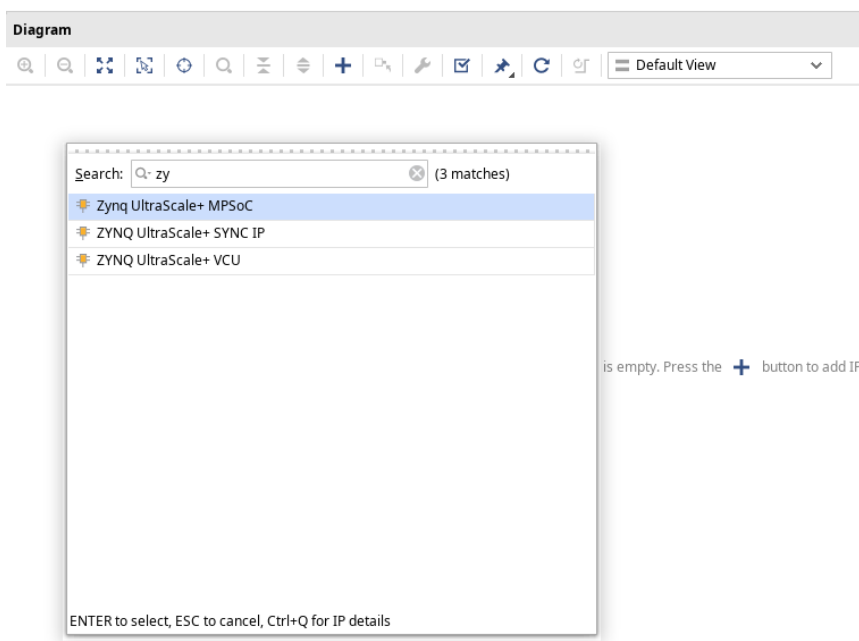


Figura 18: Paso 6.1 Vivado ML

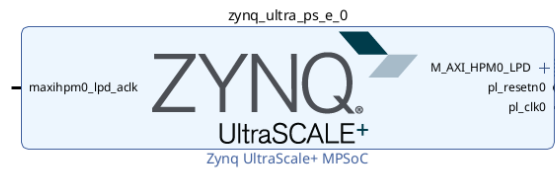


Figura 19: Paso 6.2 Vivado ML

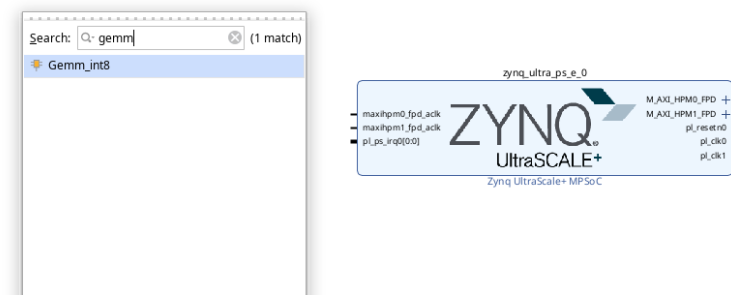


Figura 20: Paso 6.3 Vivado ML

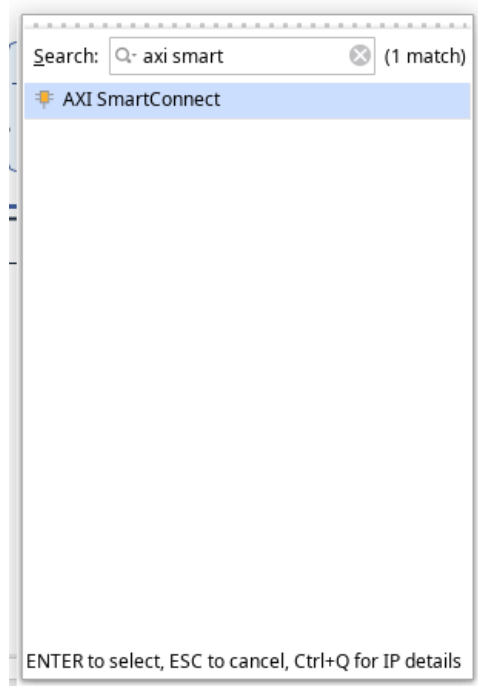


Figura 21: Paso 6.4 Vivado ML

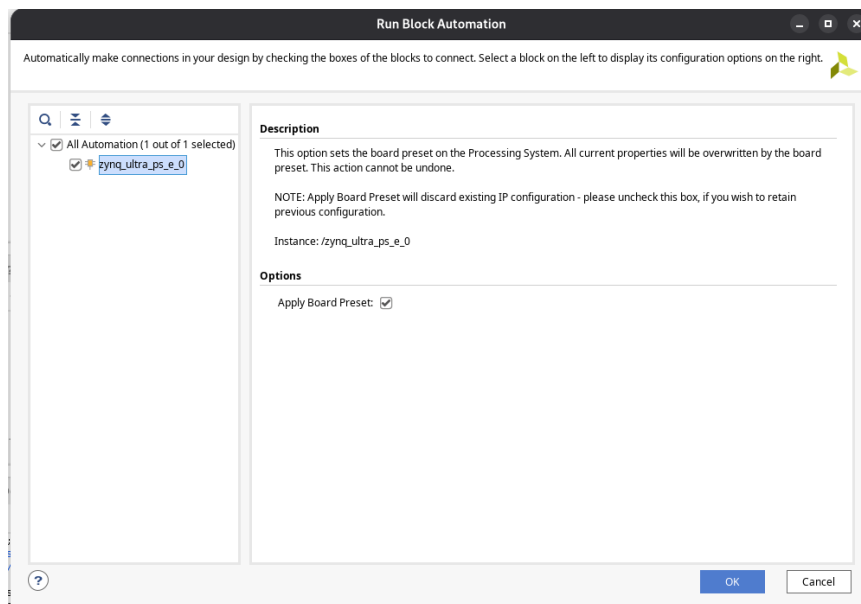


Figura 22: Paso 7 Vivado ML

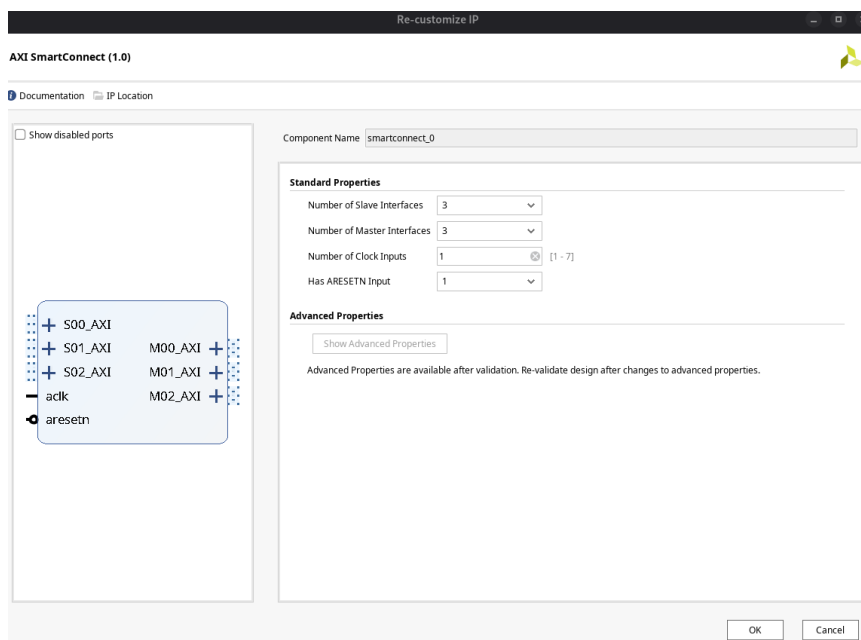


Figura 23: Paso 8 Vivado ML

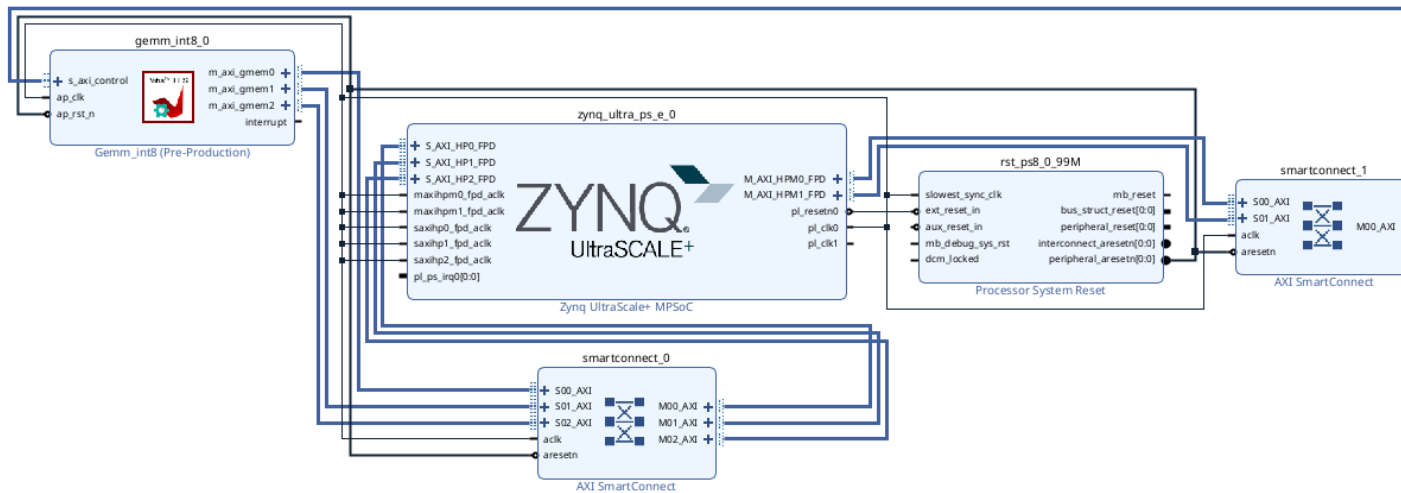


Figura 24: Paso 9 Vivado ML

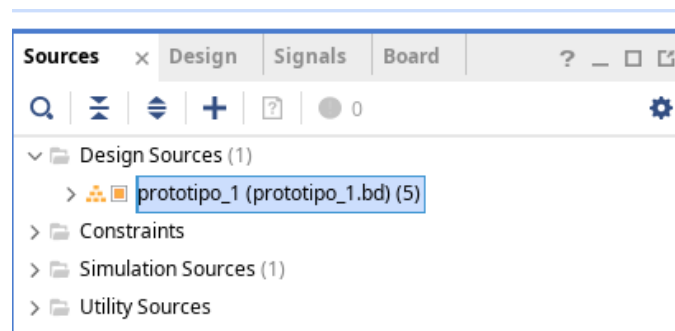


Figura 25: Paso 10 Vivado ML

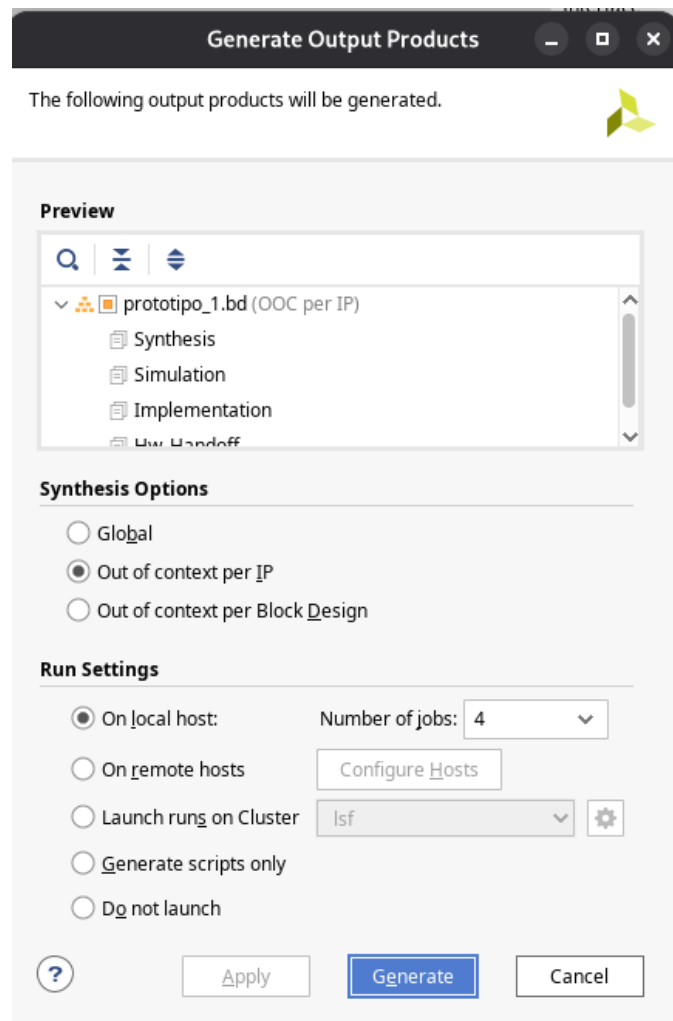


Figura 26: Paso 11 Vivado ML

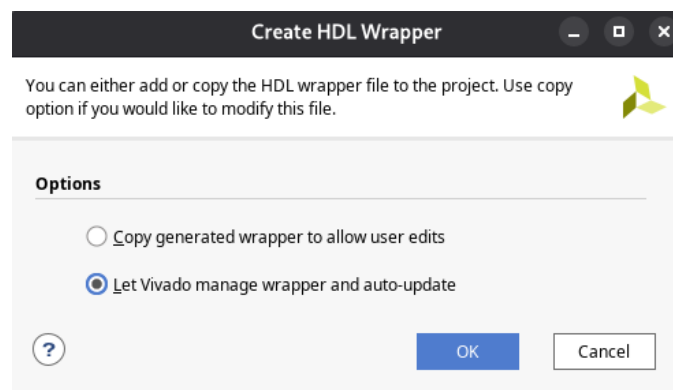


Figura 27: Paso 12 Vivado ML

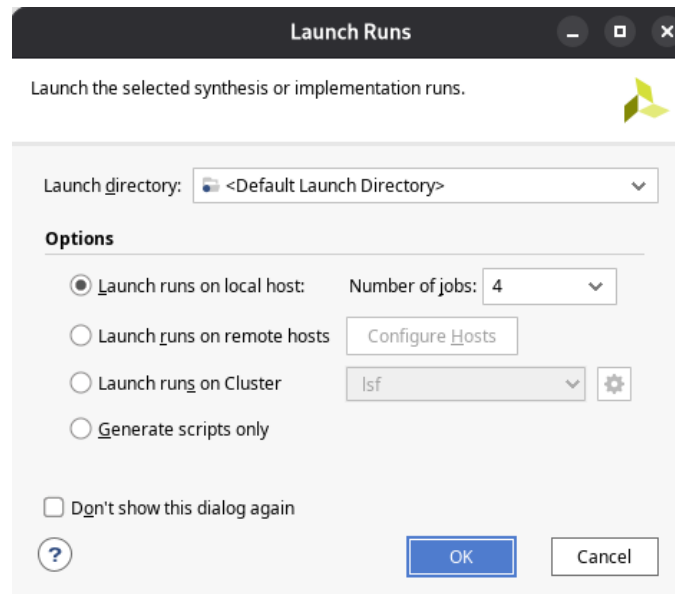


Figura 28: Paso 13.1 Vivado ML

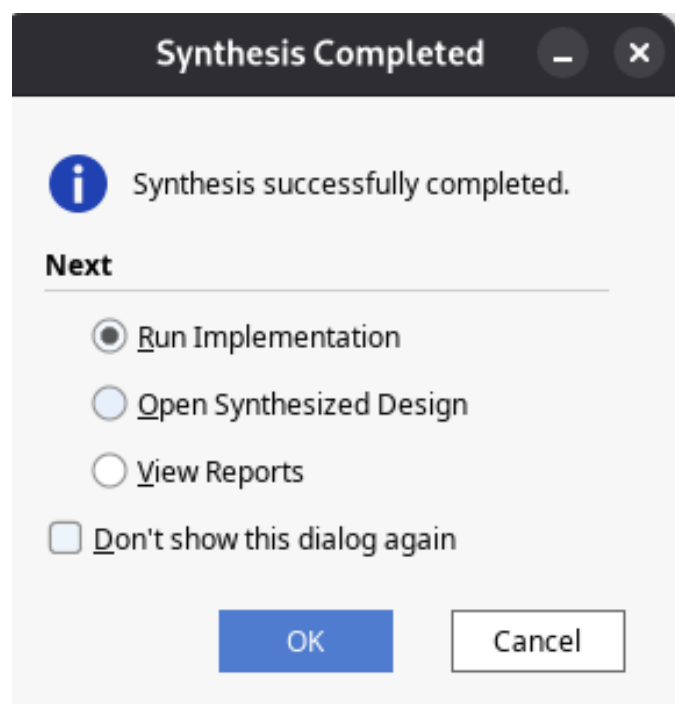


Figura 29: Paso 13.2 Vivado ML

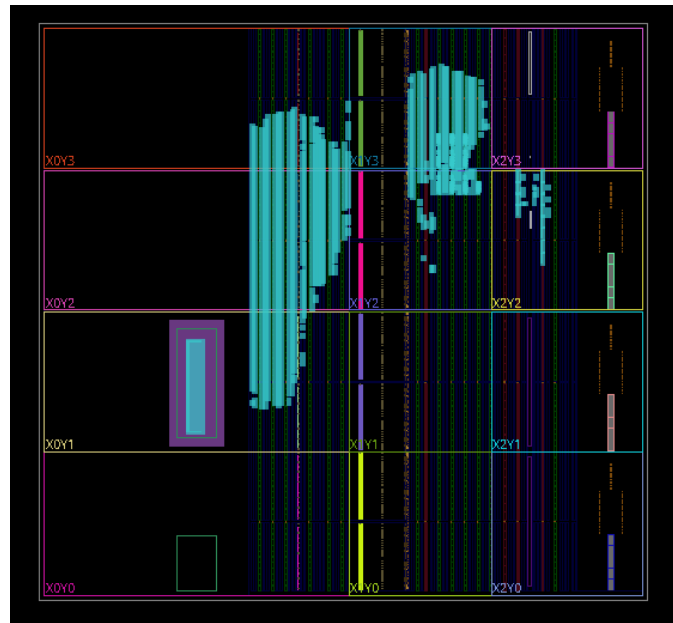


Figura 30: Paso 13.3 Vivado ML

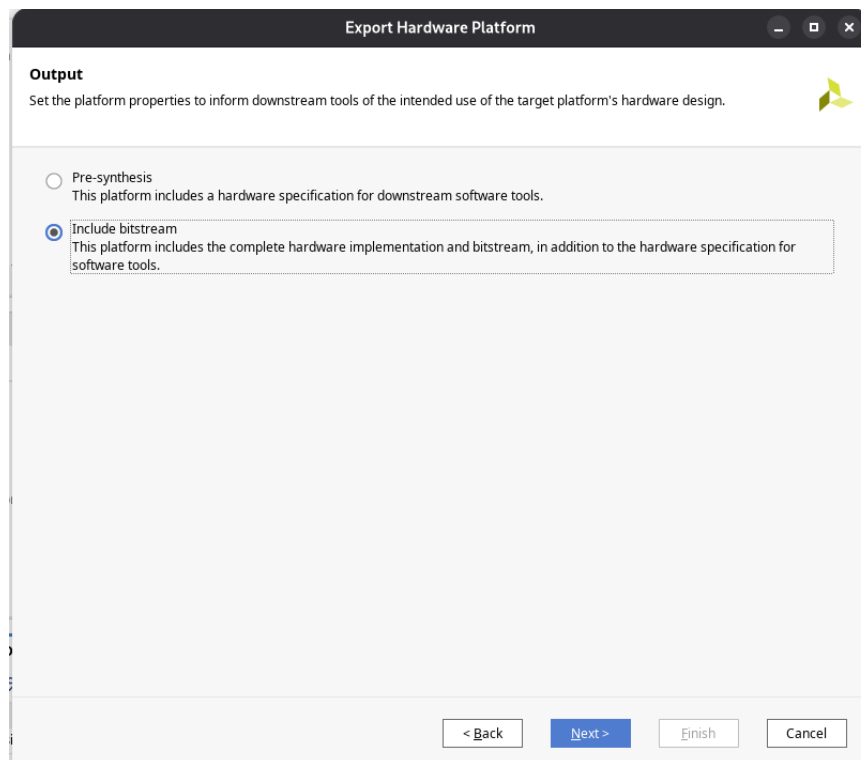


Figura 31: Paso 15 Vivado ML

New Application Project

Platform
Note: A platform project will be generated automatically in workspace for the selected XSA. It can be customized later.

☒ Select a platform from repository ☒ Create a new platform from hardware (XSA)

Hardware Specification
XSA File:

Boot Components
☒ Generate boot components
Target processor to create FSBL: ☒ psu_cortexa53_0 ☐ psu_cortexr5_0

Platform name:

Figura 32: Paso 1 Vitis ide

Application Project Details
Specify the application project name and its system project properties

Application project name:

System Project
Create a new system project for the application or select an existing one from the workspace

System project details

System project name:

Target processor

Select target processor for the Application project.

Processor	Associated applications
psu_cortexa53_0	gemm_beto_app
psu_cortexa53_1	
psu_cortexa53_2	
psu_cortexa53_3	
psu_cortexr5_0	
psu_cortexr5_1	

Show all processors in the hardware specification ☒

Figura 33: Paso 2 Vitis ide

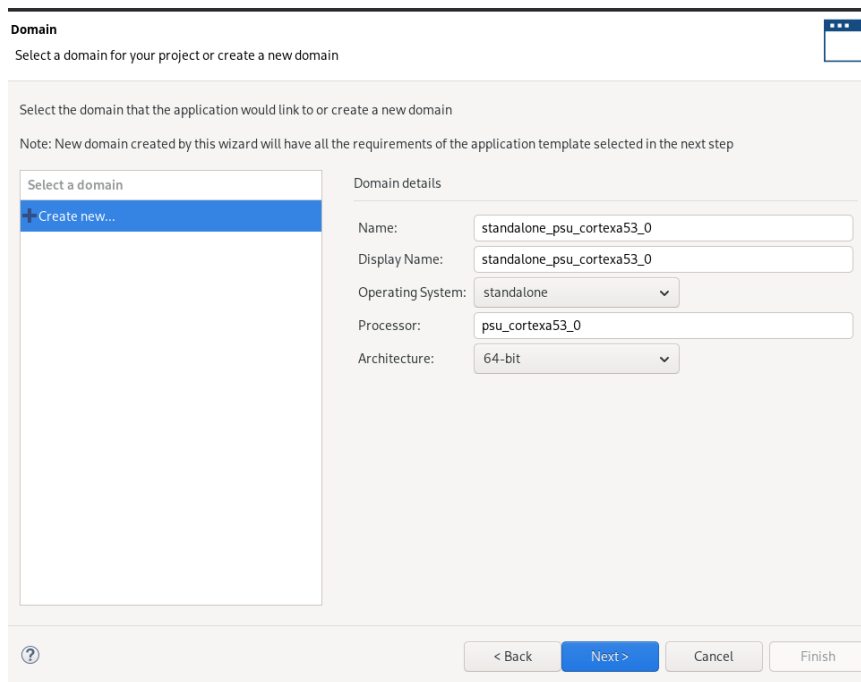


Figura 34: Paso 3 Vitis ide

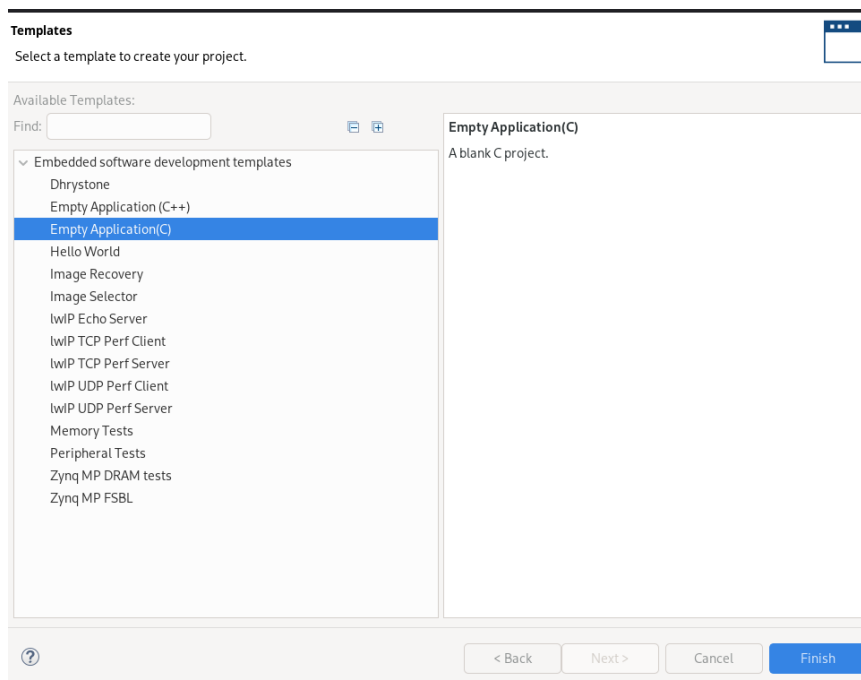


Figura 35: Paso 4 Vitis ide

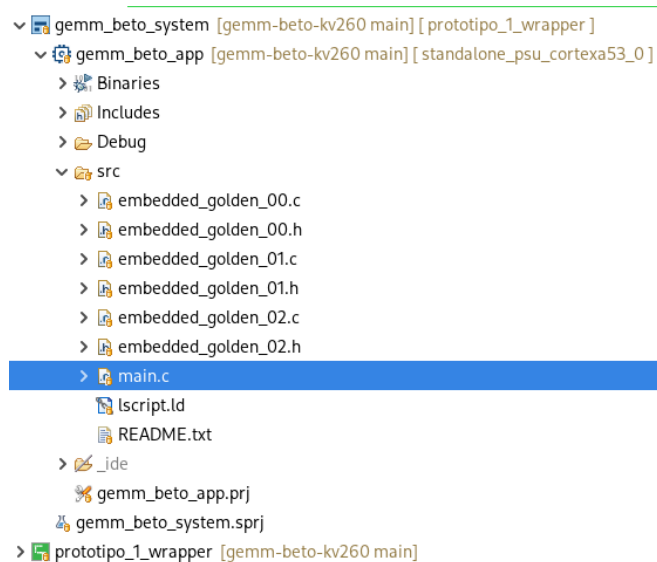


Figura 36: Paso 6 Vitis ide